

Refactoring and Command-Based Structure

This handout walks you through refactoring your ROMI robot project from a monolithic `robot.py` script into a **Command-Based** structure using `robotpy`.

Refactoring mean that you will be revising the code to make it more efficient, more readable, or generally better. Before you refactor you should always listen to the “Refactoring Song”.

<https://www.youtube.com/watch?v=SETnK2ny1R0>

Why Use the Command-Based Framework?

The Command-Based framework helps you write clean, modular, and testable robot code by organizing functionality into three key components:

- **Subsystems** represent hardware components like the drivetrain, shooter, arm, etc.
- **Commands** define behaviors that use subsystems, like “drive straight” or “turn 90 degrees.”
- The **Scheduler** runs commands at the appropriate time and ensures that commands don’t fight over control of the same subsystem.

Think of it as building blocks: each subsystem provides core capabilities, and commands tell them what to do.

Understanding the Core Methods of a Command

Every **Command** in the command-based framework follows the same structure. These methods define the command’s lifecycle.

Here’s a breakdown of the core methods you’ll typically override:

`initialize(self)`

- Called **once** when the command is first scheduled.
- Use this to reset sensors, timers, or prepare state.

Example:

```
def initialize(self):  
    self.drivetrain.resetEncoders()
```

`execute(self)`

- Called **repeatedly every 20ms** while the command is active.
- Use this to perform the main action of the command (e.g., set motor speeds).

Example:

```
def execute(self):  
    self.drivetrain.arcadeDrive(0.6, 0.0)
```

`isFinished(self) -> bool`

- Called each loop to check whether the command should end.
- Return **True** to stop the command, **False** to keep it running.

Example:

```
def isFinished(self):  
    return self.drivetrain.getAverageDistanceInch() >= 72
```

```
end(self, interrupted: bool)
```

- Called **once** when the command finishes or is interrupted.
- Use this to stop motors or clean up any side effects.

Example:

```
def end(self, interrupted):  
    self.drivetrain.arcadeDrive(0, 0)
```

Optional: `requires()` / `addRequirements()`

- Declares which subsystem(s) the command needs control of.
- Ensures that no two commands control the same subsystem at once.

Example (in constructor):

```
self.addRequirements([self.drivetrain])
```

By organizing behavior into these methods, the command-based framework makes robot logic modular, testable, and easy to reason about.

Step 0: Update `pyproject.toml` to Include `commands2`

To use the command-based framework, you'll need to install the `robotpy-commands-v2` package.

1. Uncomment the `commands2` line in your `robotpy_extras` dependencies in `pyproject.toml`:

There is stuff above this...

```
robotpy_extras = [  
    # "all",  
    # "apriltag",  
    "commands2",  
    # "cscore",  
    # "navx",  
    # "pathplannerlib",  
    # "phoenix5",  
    # "phoenix6",  
    # "rev",  
    "romi",  
    "sim",  
    # "xrp",  
]
```

2. Then run:

```
python3 -m robotpy sync
```

Make sure your development environment picks up the new library before continuing.

1. Convert Drivetrain into a Command-Based Subsystem

Step 1: Inherit from Subsystem in drivetrain.py

```
from commands2 import Subsystem

class Drivetrain(Subsystem):
    def __init__(self):
        super().__init__()
        # existing drivetrain setup code here
```

Step 2: Remove manual calls to periodic()

By making Drivetrain a wpilib.Subsystem calls are made to the periodic method every loop. This means that we don't need to do it manually in our robotPeriodic.

```
def robotPeriodic(self):
    '''This is called every cycle of the code. In general the code is loop
    through every .02 seconds.'''
    #self.drivetrain.periodic()
    pass
```

2. Create a Command to Drive Straight

Step 1: Create a new file autonomous/drivestraight.py

Step 2: Define DriveStraight as a Command class

```
from commands2 import Command

class DriveStraight(Command):
    def __init__(self, drivetrain, distance_in_inches=72):
        super().__init__()
        self.drivetrain = drivetrain
        self.distance = distance_in_inches
        self.addRequirements(drivetrain)

    def initialize(self):
        self.drivetrain.resetEncoders()

    def execute(self):
        left = self.drivetrain.getLeftDistanceInch()
        right = self.drivetrain.getRightDistanceInch()
        error = right - left
        self.drivetrain.arcadeDrive(-0.7, 0.6 * error)

    def isFinished(self):
        return self.drivetrain.getAverageDistanceInch() >= self.distance

    def end(self, interrupted):
        self.drivetrain.arcadeDrive(0, 0)
```

3. Update robot.py to Use CommandScheduler

Step 1: Import necessary command-based modules

```
from commands2 import TimedCommandRobot, CommandScheduler
```

Step 2: Replace TimedRobot with TimedCommandRobot

```
class MyRobot(TimedCommandRobot):
```

Step 3: Initialize the CommandScheduler in robotPeriodic()

```
def robotPeriodic(self):
    CommandScheduler.getInstance().run()
```

Step 4: Schedule Autonomous Command

Add another import at the top of the file.

```
from autonomous.drivestraight import DriveStraight
```

Then rewrite the autonomousInit to look like this.

```
def autonomousInit(self):
    self.autoCommand = DriveStraight(self.drivetrain)
    self.autoCommand.schedule()
```

4. Suggested Folder Structure

```
project_root/
|
>-- robot.py
>-- subsystems/
|   >-- drivetrain.py
>-- autonomous/
|   |-- drivestraight.py
```

7. Add Teleop Driving Support

Now that the robot has autonomous commands, we'll add support for **teleoperated driving** using a controller and a RunCommand.

Step 1: Import Joystick and RunCommand in robot.py

```
from wpilib import Joystick
from commands2 import RunCommand
```

Step 2: Create the controller and default drive command in robotInit()

In a command-based robot project, we often use a RunCommand to define a **default behavior** for a subsystem during teleop. This is how we tell the robot: “While no other commands are using the drivetrain, keep doing this.”

Refactor your robotInit() method to include this:

```
def robotInit(self):
    '''This method is called as the robot turns on and is often used to setup the
    joysticks and other presets.'''
    self.controller=Joystick(0)
    self.drivetrain=Drivetrain()

    self.drivetrain.setDefaultCommand(
        RunCommand(
            lambda: self.drivetrain.arcadeDrive(
```

```

        -self.controller.getRawAxis(1), # forward/backward
        self.controller.getRawAxis(0)  # rotation
    ),
    self.drivetrain
)
)

```

Then the `teleopPeriodic` can be simplified to:

```

def teleopPeriodic(self):
    '''This is called once every cycle during Teleop'''
    pass

```

Line-by-Line Explanation

`self.controller = Joystick(0)`

- Creates a `Joystick` object to read input from controller index 0 (usually a gamepad or joystick plugged in via USB).
-

`self.drivetrain.setDefaultCommand(...)`

- Sets the *default command* for the drivetrain subsystem.
 - This command will run automatically **during teleop** whenever nothing else is using the drivetrain.
-

`RunCommand(...)`

- `RunCommand` is a simple way to define a short, continuous behavior.
 - It accepts:
 1. A function to run every cycle (`lambda` in this case)
 2. A list of subsystems it requires (`self.drivetrain`)
-

The lambda

```

lambda: self.drivetrain.arcadeDrive(
    -self.controller.getRawAxis(1), # forward/backward
    self.controller.getRawAxis(0)  # rotation
)

```

- This anonymous function (a lambda) runs every ~20ms (every robot cycle).
 - It calls `arcadeDrive` with:
 - `-self.controller.getRawAxis(1)` — the Y-axis from the joystick (typically forward/backward). We negate it because pushing forward usually gives a negative value.
 - `self.controller.getRawAxis(0)` — the X-axis from the joystick (typically left/right turn).
 - This lambda replaces writing a full `Command` class for this simple behavior.
-
-

5. Next Step: Turn 90 Degrees Autonomous Command

Now that you can drive straight, your next challenge is to make the ROMI **turn 90 degrees** using the gyro sensor. The drivetrain currently exposes the Gyro, and so you can ask the drivetrain object about the how much the ROMI has turned in any of the three major axes, Turn, Pitch, and Yaw. The Turn is associated with the Z axis on the

ROMI. Write another autonomous command, and put it in `autonomous/turn90.py`, that gets the ROMI to turn exactly 90 degrees.

You will need to use a `PIDController` to make this happen, so that you don't over or under turn. Look at the `wpiLib` documentation to get a start on this:

<https://docs.wpilib.org/en/stable/docs/software/advanced-controls/controllers/pidcontroller.html>

Step 1: Create a new file `autonomous/turn90.py`

Step 2: Define `Turn90` as a `Command` class

Here is a outline of what you should do.

```
from commands2 import CommandBase

class Turn90(CommandBase):
    def __init__(self, drivetrain):
        super().__init__()
        ## Give the command access to the drivetrain in all its methods

        ## Add the requirement for this command i.e. the drivetrain
        self.addRequirements(drivetrain)

    def initialize(self):
        ## Reset the gyro

    def execute(self):
        # Determine the angle
        # determine the error
        # drive to reduce the error

    def isFinished(self):
        # Determine if you are done, being within 2 degrees is enough for me

    def end(self, interrupted):
        # Stop the movement.
```

Test this code out and refine your `error` scaling to make the turn smooth and accurate.

6. Next Steps

- Try chaining commands using `SequentialCommandGroup`.
-